LiveKit as the Communication Framework

RASSOR Project

Table of Contents

LiveKit Overview

Introduction

What is it?

LiveKit is an **open-source framework** and **cloud platform** for sharing audio, video, and data between devices (and optionally a physical AI agent). It is built on top of a production-grade ***WebRTC*** stack.

Note: WebRTC is browser-native technology for transmitting audio and video in real time. It is optimized for media delivery.

LiveKit essentially eliminates the complexity of working directly with WebRTC by:

- Managing operational and scaling challenges of WebRTC.
- **Extending its use to mobile applications**, backend services, and telephony integrations.

Building Blocks

LiveKit's architecture is built around three core concepts that work together to enable realtime communication:

Concept	Description	Key capabilities
➔ *Rooms*	Virtual spaces where participants connect and communicate.	Create, list, and delete rooms.
➔ *Participants*	The entities that join rooms.	List, remove, and mute participants.
➔ *Tracks*	Media streams that participants publish and subscribe to. LiveKit supports audio tracks, video tracks, and data tracks.	Publish camera, microphone, and screen share tracks.

How Does it Work?

LiveKit server

An open-source WebRTC ***Selective Forwarding Unit***[1] that orchestrates real time communication between users and, optionally, agents. The server handles:

- Signaling.
- ***Network Address Translation*** (NAT) traversal.
- ***RPT***[2] routing.
- Adaptive degradation.
- Quality-of-service controls.

SDKs *and clients*

Native SDKs provide a consistent programming model.

Integration services.

LiveKit supports:

- Recording and streaming (Egress).
- External media streams (Ingress).
- Integration with SIP[3], PSTN[4], and other communication systems (telephony).
- Server APIs for programmatic session management.

[1] A Selective Forwarding Unit (SFU) is a type of media server used in WebRTC that selectively forwards streams to each participant, reducing CPU load.
[2] Real-time Transport Protocol (RTP) is the standard protocol for delivering audio and video over IP networks.
[3] Session Initiation Protocol (SIP) is a signaling protocol used to establish, modify, and terminate multimedia sessions.
[4] Public Switched Telephone Network (PSTN) is the traditional circuit-switched telephone network.

Implementation Workflow

Connecting to LiveKit

Connection is established through a `room` object.

> A `room` is a core concept that represents an **active LiveKit session**.

Participants can be users, AI agents, devices, or other programs. Each participant can publish audio, video, and data, and can selectively subscribe to tracks published by others.

SDKs

LiveKit SDKs provide a **unified API** for joining rooms, managing participants, and handling media tracks and data channels.

LiveKit includes open source SDKs for every major platform including *JavaScript*, *Swift*, *Android*, ***React Native***, *Flutter*, and *Unity*.

LiveKit also has SDKs for realtime backend apps in ***Python***, *Node.js*, *Go*, *Rust*, *Ruby*, and *Kotlin*.

RASSOR Context

> In the context of the RASSOR project, the React Native mobile application would join (or create) a room by using LiveKit's SDK.
>
> To install the SDK for a React Native application, the following command must be executed:
>
> ```
> npm install @livekit/react-native
> @livekit/react-native-webrtc livekit-client
> ```

Connecting to a room

Key concepts:

- Rooms are created automatically when the first participant joins, and they are deleted the moment the last participant leaves.
- Rooms are identified by name.
- Participants must ues an identity, which might be a unique string.

Workflow:

1. Each participant must have these paramenters:
    a. `wsUrl`: The websockets URL of a LiveKit server.
    b. `token`: A unique access token.
        i. It encodes the room name, participant's identity, and permissions.
2. After connection is established, the `room` object contains two key attributes:
    a. `localParticipant`: An object that represents the current user.
    b. `remoteParticipants`: A map containing other participants in the room, keyed by their identity.
3. Each participant publishes and subscribes to realtime media tracks, or exchanges data with other participants.

Disconnecting from a room

Disconnection may happen in two ways:

- Calling `Room.disconnect()`.
- Automatic disconnection due to:
    - Exiting the app without calling `disconnect()`.
    - DUPLICATE_IDENTITY: Disconnected because another participant with the same identity joined the room.
    - ROOM_DELETED: The room was closed via the `DeleteRoom` API.
    - PARTICIPANT_REMOVED: Removed from the room using the `RemoveParticipant` API.
    - JOIN_FAILURE: Failure to connect to the room, possibly due to network issues.
    - ROOM_CLOSED: The room was closed because all participants left[5].

[5] All of these disconnection reasons are provided by the *Disconnected* event.

Advantages of Using LiveKit

Events Handling

LiveKit emits a number of events in these situations:

- On the `Room` object, such as when new participants join or tracks are published.
- When a participant gets disconnected from a room due to server-initiated actions. In such cases, a `Disconnected` event is emitted, providing one of the reasons described in the last section (see “Disconnecting from a room”).
- When a full reconnection is required, a `reconnecting` event is triggered.

Connection Reliability

LiveKit enables reliable connectivity in a wide variety of network conditions. It tries the following WebRTC connection types in descending order:

1. ICE over UDP: ideal connection type, used in the majority of conditions.
2. TURN with UDP (3478): used when ICE/UDP is unreachable.
3. ICE over TCP: used when the network disallows UDP (i.e. over VPN or corporate firewalls).
4. TURN with TLS: used when the firewall only allows outbound TLS connections.

When a user’s network connection gets interrupted, LiveKit attempts to resume the connection automatically.

Implementation Overview

LiveKit can be implemented to handle the underlying WebRTC logic of our communication software. According to LiveKit's documentation:

> ***LiveKit transport*** *provides the foundation for building realtime applications using WebRTC. It includes client and server SDKs for multiple platforms, comprehensive media and data handling, stream export and import services, and hardware integration capabilities. Together, these components enable you to build production-ready realtime applications that work across web, mobile, hardware, and* ***embedded devices****.*

In this section, LiveKit's implementation to our specific project is briefly described. The purpose is to clarify its purpose and implementation workflow, as well as to identify potential challenges.

Realtime SDKs

Realtime SDKs are required in order to connect to a room and participate in realtime communication. These SDKs manage:

- WebRTC connections with automatic reconnection and quality adaptation.
- Media capture.
- Room management.
- Track handling: subscribe to and publish audio and video tracks.
- Data channels.

There SDKs for the following platforms:

- JavaScript.
- iOS/macOS/visionOS.
- Android.
- Flutter.
- **React Native**.
- Unity.
- **C++**.

> ***Note**: LiveKit also supports specialized platforms and use cases beyond the main web and mobile platforms:*
>
> - *Rust SDK: For systems programming and **embedded applications**.*
> - *ESP32: For **IoT** and **embedded devices**.*

Cloud Hosting vs Self-Hosting

When building with LiveKit it is possible to either self-host the open-source server or use the managed LiveKit Cloud service:

	Self-Hosted	**LiveKit Cloud**
➔ *Realtime media*	Full support	Full support
➔ *Egress*	Full support	Full support
➔ *Ingress*	Full support	Full support
➔ *Built-in interface*	N/A	Included
➔ *Who manages it*	The user	LiveKit
➔ *Architecture*	Single-home SFU	Global mesh SFU
➔ *Connection model*	Single server per room	Each user connects to the nearest edge

---

> ***Note**: Both cloud hosting and self-hosting run the same [open-source LiveKit server] and support the same APIs and SDKs, which means that you can move from one implementation to the other without rewriting existing code.*

LiveKit Cloud

The LiveKit cloud combines realtime audio, video, and data streaming with production-grade observability in a single, cohesive platform.

It includes:

- **Realtime communication core**: A fully managed, **globally distributed** mesh of LiveKit servers that powers **low-latency** audio, video, and data streaming for realtime applications.
- Managed agent hosting, built-in interface, and native telephony: For AI agents development.
- **Observability and operations**: production-grade **analytics**, **logs**, and **quality metrics**.

Advantages:

- End-to-end platform.
- Zero operational overhead: No need to manage servers, scaling, or infrastructure.
- Global edge network: Users connect to the closest region for minimal latency.
- Elastic, unlimited scale.
- Enterprise-grade reliability.
- Comprehensive analytics.
- Seamless developer experience.

Pricing (including only the two most feasible plans):

Plan	Cost	Key Features
➔ *Free Tier*	$0/month	10,000 participant minutes per month, 1 free phone number, global edge network, analytics, community support.
➔ *Ship*	Starting at $50/month	Adds team collaboration, instant rollback, email support.

The free tier is generous for development and small projects (like in our case). 10,000 minutes provides roughly **166 hours of 2-participants calls per month**.

Self-Hosting

Self-hosting enables LiveKit deployment on a **customized infrastructure**, whether for **local development**, **production deployments** on **virtual machines** or **Kubernetes**, or distributed **multi-region setups**.

Topics of interest:

- **Running locally**: Get LiveKit running locally.
- **Deployment**: Deploy LiveKit servers to production.
- **Virtual machines**: Deploy LiveKit servers on VMs.
- **Kubernetes**: Deploy LiveKit servers on Kubernetes clusters.
- **Distributed multi-region**: Deploy LiveKit servers across multiple regions.

Conclusions

As seen throughout the document, implementing LiveKit poses several advantages:

- Easier connection establishment.
- Automatic reconnection strategies.
- High customization.
- A wide variety of Software Development Kits (SDKs).
- A reliable, redundant connection.

LiveKit completely eliminates the necessity of setting up a signaling server, connecting to STUN servers for ICE candidates generation and TURN servers for relaying data, setting up media streams and data channels, and in general managing connection establishment and termination. In addition to this, implementing LiveKit into the RASSOR project would allow for AI agents integration and multiple human operators to collaborate in real time.

Moreover, LiveKit integrates seamlessly with both React Native (for the mobile application participant) and C++ (the single-board computer participant) through their SDK suite, thus reducing the complexity of using WebRTC by a great degree.

Additionally, LiveKit's flexibility in deployment alternatives makes it feasible to reduce costs as needed. Both the cloud hosted and the self-hosted options offer significant advantages, and even though self-hosting the LiveKit server would provide greater flexibility and zero cost (apart from that of mounting and maintaining the infrastructure), the complexity of setting it up locally surpasses its advantages. Also, LiveKit's cloud free tier seems to be a reasonable option since it isn't expected that our project will need to be scaled up too much.

In summary, LiveKit should be considered as a serious option for developing the communication software, and further research and experimentation must be done to confirm that it's the best fit for our project.

Bibliography

References

Connecting to LiveKit. (n.d.). LiveKit Documentation. Retrieved April 5, 2026, from https://docs.livekit.io/intro/basics/connect/

Introduction. (n.d.). LiveKit Documentation. Retrieved April 5, 2026, from https://docs.livekit.io/transport/

LiveKit Cloud. (n.d.). LiveKit Documentation. Retrieved April 5, 2026, from https://docs.livekit.io/intro/cloud/

LiveKit Pricing. (n.d.). LiveKit. Retrieved April 5, 2026, from https://livekit.com/pricing

Rooms, participants, and tracks overview. (n.d.). LiveKit Documentation. Retrieved April 5, 2026, from https://docs.livekit.io/intro/basics/rooms-participants-tracks/

Self-hosting overview. (n.d.). LiveKit Documentation. Retrieved April 5, 2026, from https://docs.livekit.io/transport/self-hosting/

Understanding LiveKit overview. (n.d.). LiveKit Documentation. Retrieved April 5, 2026, from https://docs.livekit.io/intro/basics/